SIMATS ENGINEERING

Engineer to Excel

SIMATS

Saveetha Institute of Medical And Technical Sciences

(Declared as Deemed to be University under Section 3 of UGC Act 1956)

PROGRAMMING IN JAVA - CSA0901

CAPSTONE PROJECT PRESENTATION

Student Attendance Management System

Guided by:

Dr. B Shyamala Gowri

Presented by:

Sanjay S - 1972260017

Pranav Adarsh - 1924260042

Jaya Surya - 1924260172

S Darshan - 1925260086

Praveen - 1921260144

Engineer to Excel

INTRODUCTION

- Attendance is an important part of every school and college.
- Manual attendance using registers takes more time.
- It is difficult to maintain attendance record manually.
- An Attendance Management System stores attendance digitally.
- The system is developed using Java.
- It helps teachers manage attendance quickly and accurately.

Engineer to Excel

ABSTRACT

- The Attendance Management System is a Java-Based application designed to manage student attendance efficiently.
- The system allow teachers to add student details, mark attendance, view attendance records, and generate reports.
- It reduces paperwork, saves time, and minimizes errors.
- The project demonstrates the use of Java concepts such as Classes, Object, Loops, Arrays or ArrayList, Methods and File Handling.

Engineer to Excel

OBJECTIVE & SDG GOAL

OBJECTIVES

- Develop a simple Java based Attendance Management System.
- Store and manage student attendance records digitally.
- Reduce manual paperwork and human errors.
- Monitor student attendance accurately.
- Generate attendance reports quickly.
- Improve classroom administration and educational efficiency.
- Encourage regular student attendance and participation.

PROJECT OBJECTIVE

To develop a Java based Attendance Management System that helps educational institutions to record, manage and monitor student attendance efficiently and accurately, reducing manual work and improving the quality of education management.

HOW OUR PROJECT SUPPORTS SDG 4

- Maintains accurate attendance records.
- Helps teachers monitor student attendance efficiently.
- Reduces manual paperwork and human errors.
- Identifies students with low attendance.
- Encourages regular student participation.
- Improves the quality and efficiency of educational management.

CONTRIBUTION TO SDG 4 – QUALITY EDUCATION

- Promotes digital education management
- Improves attendance tracking
- Supports teachers with efficient record management
- Helps identify students with low attendance
- Contributes to better learning outcomes and educational quality

The Attendance Management System contributes to UN Sustainable Development Goal 4 (Quality Education) by providing an efficient, accurate and reliable method for managing student attendance.

Engineer to Excel

PROBLEM STATEMENT

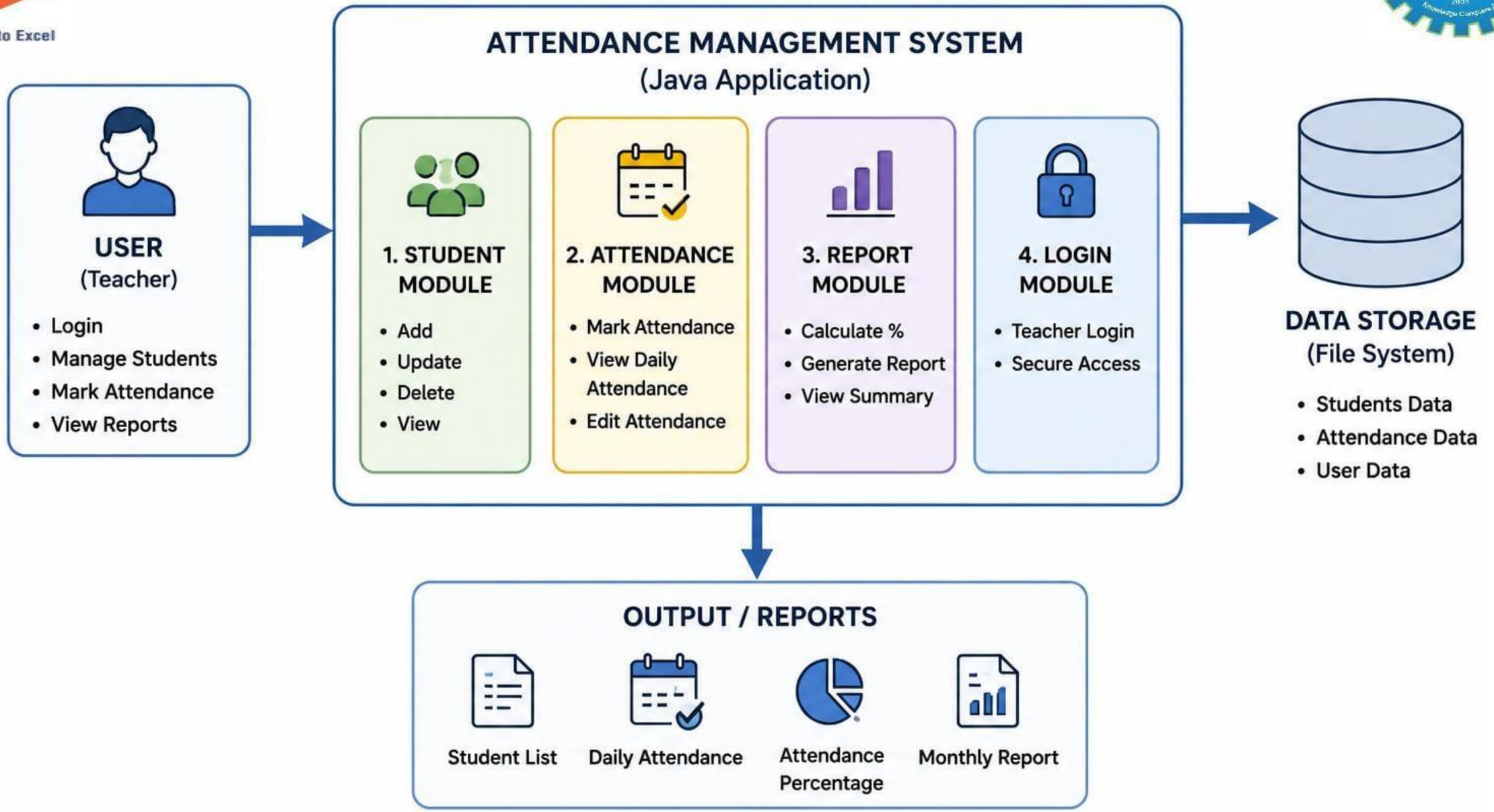
- Manual attendance consumes time.
- Attendance registers can be lost or damaged.
- Calculating attendance percentage manually is difficult.
- Human errors may occur.
- There is a need for a simple digital attendance system.

Engineer to Excel

LITERATURE REVIEW

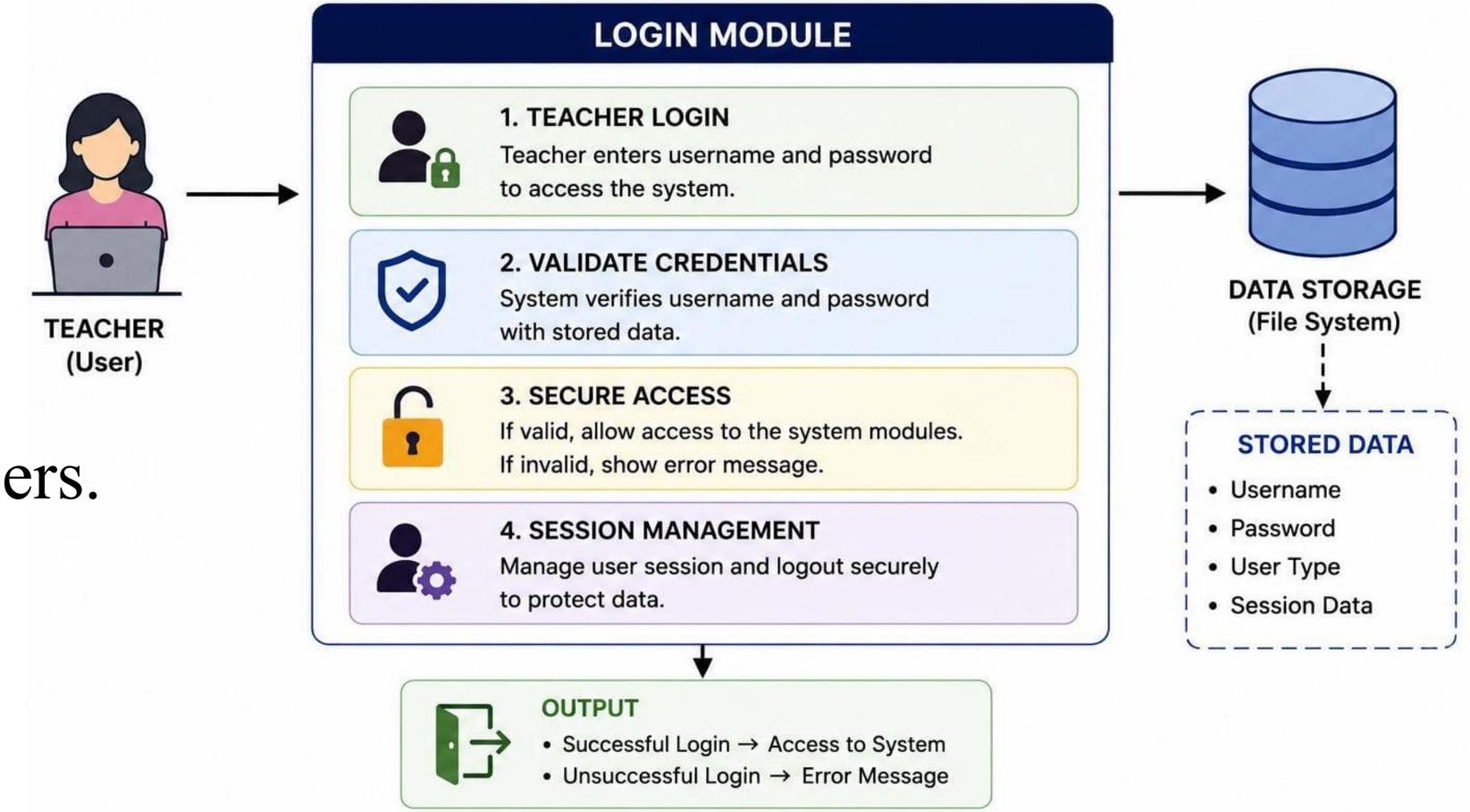
YEAR	AUTHOR(S)	SYSTEM	TECHNOLOGY	LIMITATIONS
2023	Sharma	Attendance App	Java	Desktop only
2024	Kumar	Student Management	Java + MySQL	Complex setup
2024	Patel	Attendance System	Java Swing	No mobile support

SYSTEM ARCHITECTURE



MODULE 1 — Login System

- Teacher Login.
- Username and Password.
- Secure access.
- Prevent Unauthorized Users.



Module 1 - Source Code

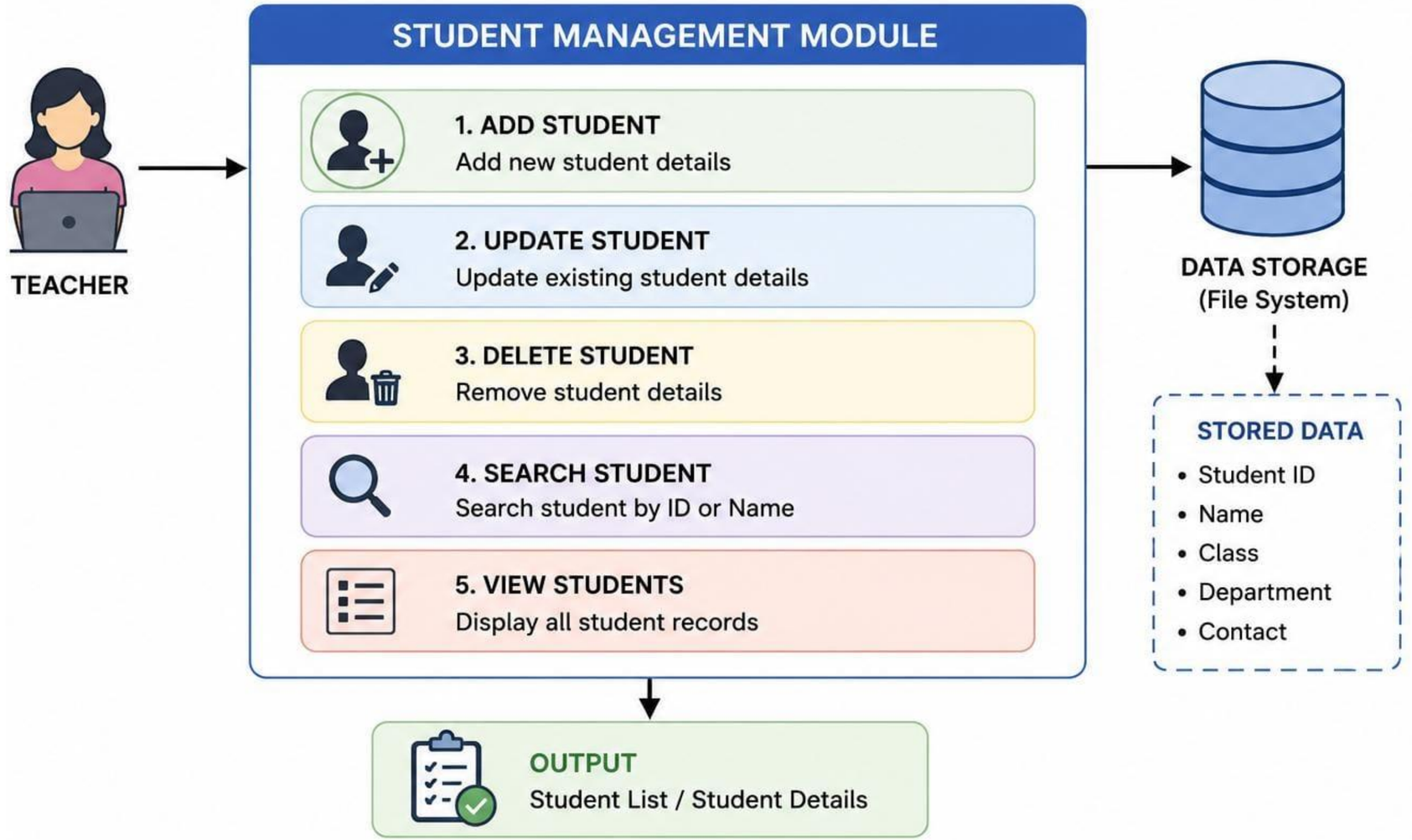
Engineer to Excel

```
/* ============================================================
   USER MANAGEMENT
 ============================================================ */
function getUsers() {
    return readStore(DB.USERS);
}

function saveUsers(users) {
    writeStore(DB.USERS, users);
}

/* ============================================================
   REGISTER USER
 ============================================================ */
function registerUser(name, username, password) {
    const users = getUsers();

    if (users.some(user =>
        user.username.toLowerCase() ===
        username.toLowerCase()
    )) {
        return {
            ok: false,
            message: "That username is already taken."
        };
    }
    /*
       Demo project only.
       Passwords are stored in localStorage.
       A real production system should use
       backend authentication and password hashing.
     */
    users.push({
        name,
        username,
        password
    });

    saveUsers(users);

    return { ok: true };
}
```

```
/* ============================================================
   LOGIN USER
 ============================================================ */
function loginUser(username, password) {
    return getUsers().find(user =>
        user.username.toLowerCase() ===
        username.toLowerCase()
        &&
        user.password === password
    ) || null;
}

/* ============================================================
   CURRENT USER
 ============================================================ */
function currentUser() {
    try {
        return JSON.parse(
            sessionStorage.getItem("sams_currentUser")
        );
    }
    catch (e) {
        return null;
    }
}

/* ============================================================
   SET CURRENT USER
 ============================================================ */
function setCurrentUser(user) {
    sessionStorage.setItem(
        "sams_currentUser",
        JSON.stringify(user)
    );
}
```

MODULE 2 — Student Management

- Add Student.
- Update Student.
- Delete Student.
- Search Student.
- Store Student details.



MODULE 2 — Source Code

Engineer to Excel

DATABASE

```
// Storage key for students
const STUDENT_STORAGE_KEY = "sams_students";
```

STORE STUDENT DETAILS

```
// Read student data from localStorage
function getStudents() {
   try {
      return JSON.parse(
        localStorage.getItem(STUDENT_STORAGE_KEY)
      ) || [];

   } catch (error) {
      return [];
   }
}

// Save student data to localStorage
function saveStudents(students) {
   localStorage.setItem(
      STUDENT_STORAGE_KEY,
      JSON.stringify(students)
   );
}
```

GENERATE STUDENT ID

```
function generateStudentID() {
   return "STU-" +
      Math.random()
        .toString(36)
        .substring(2, 8)
        .toUpperCase();
}
```

1. ADD STUDENT

```
function addStudent(student) {
   const students = getStudents();

   // Generate unique Student ID
   student.id = generateStudentID();

   // Add student to the list
   students.push(student);

   // Store updated list
   saveStudents(students);

   return student;
}
```

2. UPDATE STUDENT

```
function updateStudent(id, updatedData) {
   const students = getStudents();

   // Find student using ID
   const index = students.findIndex(
      student => student.id === id
   );

   // Student not found
   if (index === -1) {
      return false;
   }

   // Update existing student details
   students[index] = {
      ...students[index],
      ...updatedData
   };

   // Store updated list
   saveStudents(students);
   return true;
}
```

3. DELETE STUDENT

```
function deleteStudent(id) {
   const students = getStudents();

   // Remove student with matching ID
   const updatedStudents = students.filter(
      student => student.id !== id
   );
   // Store updated list
   saveStudents(updatedStudents);
   return true;
}
```

4. SEARCH STUDENT

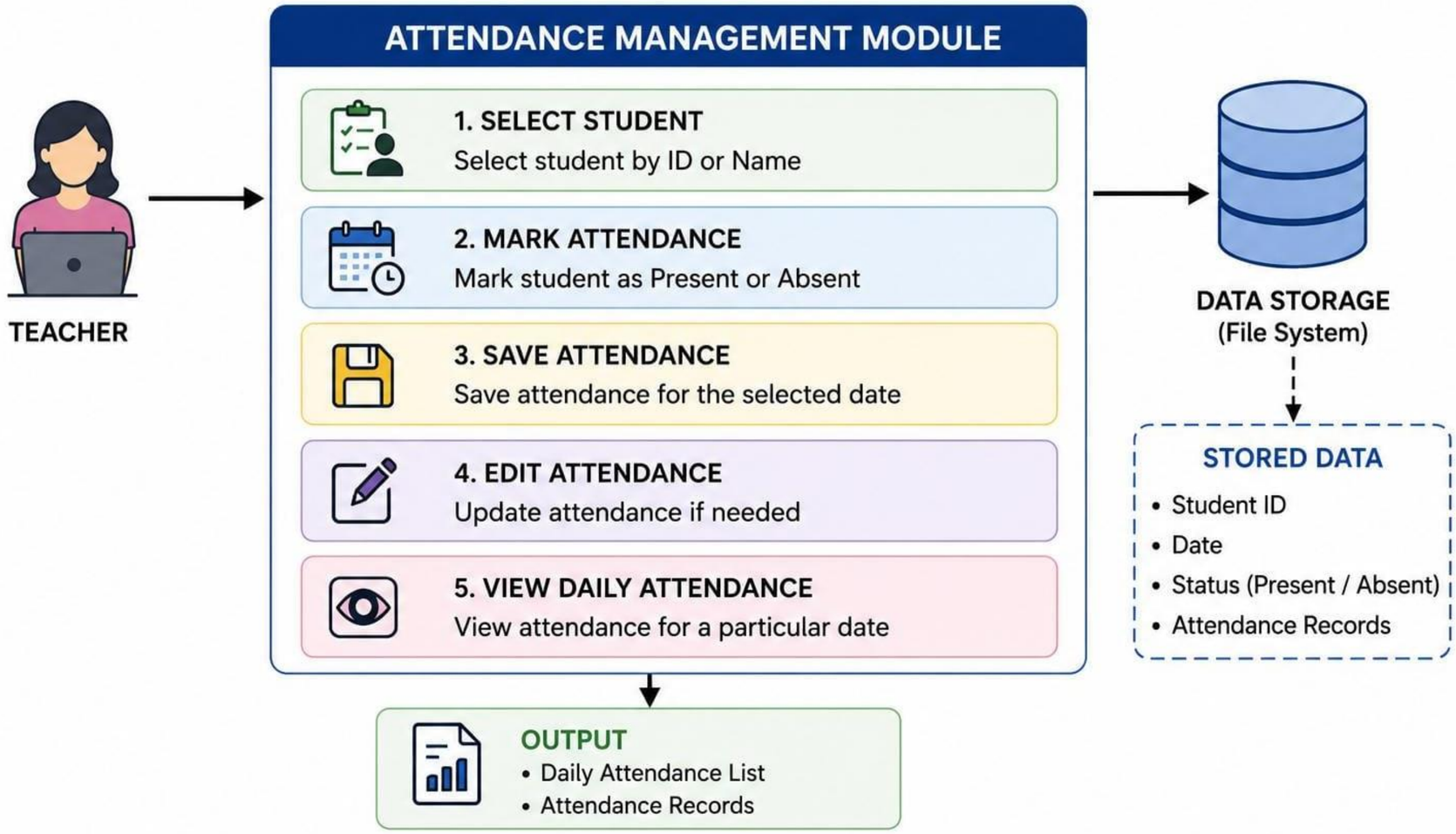
```
function searchStudents(query) {
   const students = getStudents();

   // Remove extra spaces and convert to lowercase
   const searchText = query.trim().toLowerCase();

   // If search box is empty, return all students
   if (!searchText) {
      return students;
   }

   // Search by student name or student ID
   return students.filter(student =>
      student.name.toLowerCase().includes(searchText) ||
      student.id.toLowerCase().includes(searchText)
   );
}
```

VIEW ALL STUDENTS

```
function getAllStudents() {
   return getStudents();
}
```

MODULE 3 — Attendance Management

- Select Student.
- Mark Present or Absent.
- Save Attendance.
- View Daily Attendance.
- Edit Attendance.



MODULE 3 — Source Code

Engineer to Excel

1 GET ATTENDANCE DATA

```
// Read attendance data from storage
function getAttendance() {
   return readStore(DB.ATTENDANCE);
}
```

2 STORE ATTENDANCE DATA

```
// Save attendance data to storage
function saveAttendance(list) {
   writeStore(DB.ATTENDANCE, list);
}
```

3 SELECT STUDENT

```
// Find student by ID
function selectStudent(studentId) {
   const student = getStudents().find(
       student => student.id === studentId
   );
   return student || null;
}
```

4 MARK PRESENT OR ABSENT

```
// Mark attendance (create new or update existing)
function markAttendance(studentId, date, status) {
   const records = getAttendance();

   // Check whether attendance already exists
   const index = records.findIndex(
      record => record.studentId === studentId &&
         record.date === date
   );
   // If no record exists, create a new one
   if (index === -1) {
      records.push({
         studentId: studentId,
         date: date,
         status: status
      });
   }
   // If record already exists, update it
   else {
      records[index].status = status;
   }
   saveAttendance(records);
}
```

5 SAVE ATTENDANCE

```
// Save attendance for multiple students on a given date
function saveDailyAttendance(date, statusMap) {
   Object.entries(statusMap).forEach(
      ([studentId, status]) => {
        markAttendance(
           studentId,
           date,
           status
        );
      }
   );
}
```

6 VIEW DAILY ATTENDANCE

```
// Get all attendance records for a specific date
function getAttendanceByDate(date) {
   return getAttendance().filter(
      record => record.date === date
   );
}
```

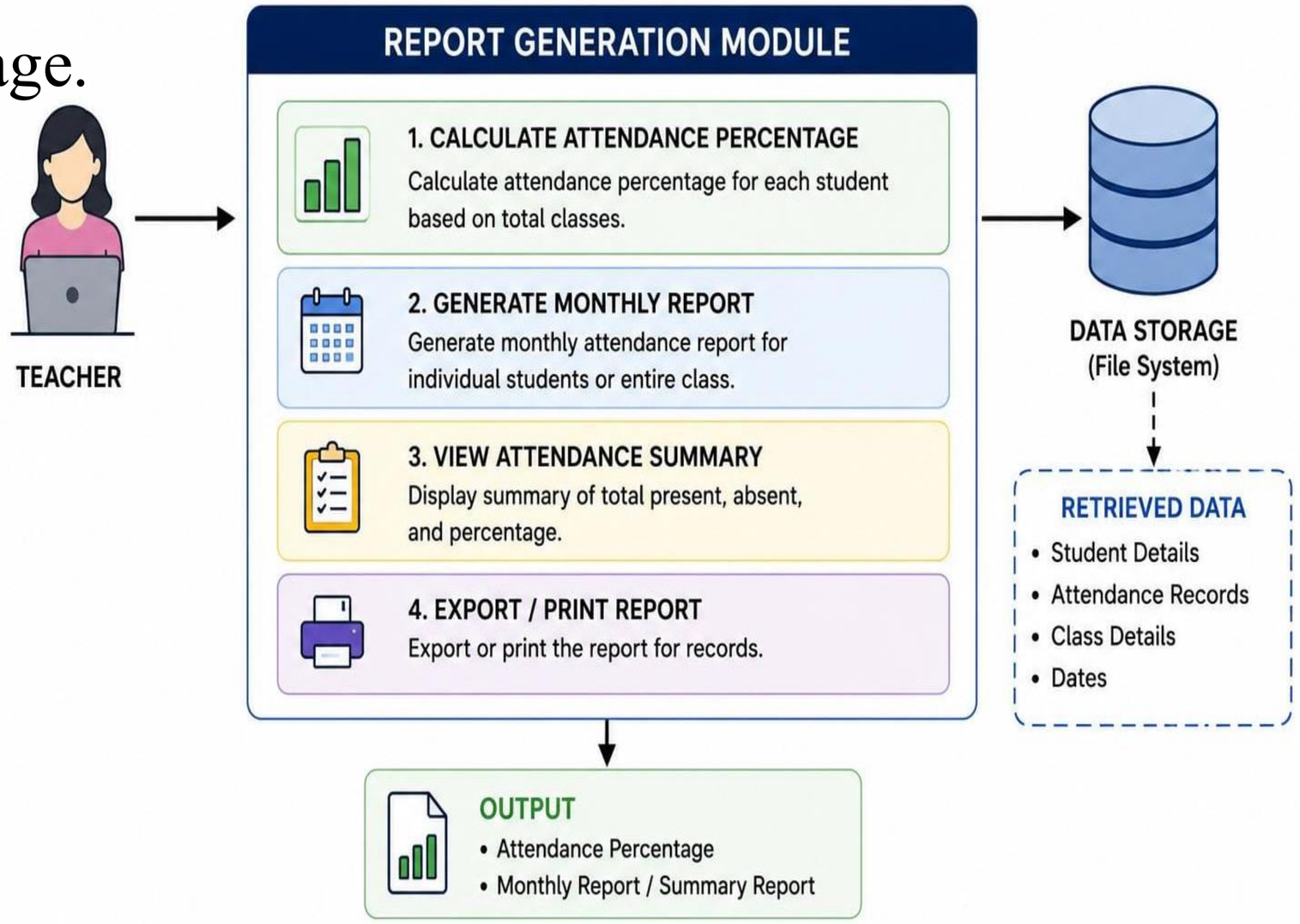
7 EDIT ATTENDANCE

```
// Edit existing attendance record
function editAttendance(studentId, date, newStatus) {
   const records = getAttendance();

   const index = records.findIndex(
      record =>
         record.studentId === studentId &&
         record.date === date
   );

   if (index === -1) {
     return false;   // Record not found
   }

   records[index].status = newStatus;
   saveAttendance(records);
   return true;     // Update successful
}
```

LEGEND: function / const · return / null / false / true · method / variable · comment · string / value · operator · punctuation

MODULE 4 — Report Generation

- ❖Calculate Attendance Perdentage.
- ❖Generate Monthly Report.
- ❖Display Attendance Summary.
- ❖View Individual Student Attendance.



MODULE 4 — Source Code

Engineer to Excel

1 CALCULATE ATTENDANCE PERCENTAGE

```
function calcAttendancePercentage(studentId) {
  const records = getAttendanceByStudent(studentId);

  // No attendance records
  if (records.length === 0) {
     return 0;
  }

  const present = records.filter(
     record => record.status === "Present"
  ).length;

  return Math.round(
     (present / records.length) * 100
  );
}
```

2 GENERATE ATTENDANCE SUMMARY

```
function generateSummaryReport() {
  return getStudents().map(student => {
      const records = getAttendanceByStudent(student.id);

      const present = records.filter(
          record => record.status === "Present"
      ).length;

      const absent = records.filter(
          record => record.status === "Absent"
      ).length;

      return {
          ...student,
          total: records.length,
          present: present,
          absent: absent,
          percentage: records.length
              ? Math.round((present / records.length) * 100)
              : 0
      };
  }));
}
```

3 GENERATE MONTHLY REPORT

```
function generateMonthlyReport(yearMonth) {
  const records = getAttendance().filter(
     record => record.date.startsWith(yearMonth)
  );

  return getStudents().map(student => {
     const ownRecords = records.filter(
        record => record.studentId === student.id
     );
     const present = ownRecords.filter(
         record => record.status === "Present"
     ).length;
     const absent = ownRecords.filter(
         record => record.status === "Absent"
     ).length;
     return {
         ...student,
         total: ownRecords.length,
         present: present,
         absent: absent,
         percentage: ownRecords.length
             ? Math.round((present / ownRecords.length) * 100)
             : 0
     };
  });
}
```

4 VIEW INDIVIDUAL STUDENT ATTENDANCE

```
function getIndividualStudentAttendance(studentId) {
  const student = getStudents().find(student => student.id === studentId);

  if (!student) {
     return null;
  }

  const records = getAttendanceByStudent(studentId);
  const present = records.filter(record => record.status === "Present").length;
  const absent = records.filter(record => record.status === "Absent").length;

  return {
     student: student,
     records: records,
     total: records.length,
     present: present,
     absent: absent,
     percentage: records.length
         ? Math.round((present / records.length) * 100)
         : 0
  };
}
```

LEGEND: function / keyword · method / object · string · comment · number · operator · punctuation

Future Scope

- Face Recognition Attendance.
- QR Code Attendance.
- Fingerprint Attendance.
- Mobile App Integration.
- Cloud Database.
- SMS Notification to Parents.
- AI-Based Attendance Analytics.

Engineer to Excel

CONCLUSION

- The Attendance Management System simplifies attendance management by replacing manual records with a digital system. It saves time, improves accuracy, and helps teachers manage student attendance efficiently. The project also demonstrates the practical application of Java programming concepts.

Engineer to Excel

REFERENCES

1. Herbert Schildt, Java: The Complete Reference, 12th Edition, McGraw-Hill Education. https://www.mhprofessional.com/java-the-complete-reference-twelfth-edition

2. E. Balagurusamy, Programming with Java, McGraw-Hill Education.https://www.mheducation.co.in/programming-with-java

3. Oracle Java Documentation https://docs.oracle.com/javase/

4. Oracle Java Tutorials https://dev.java/learn/

5. GeeksforGeeks – Java Programming Language https://www.geeksforgeeks.org/java/

6. W3Schools – Java Tutorial https://www.w3schools.com/java/

7. TutorialsPoint – Java Tutorial https://www.tutorialspoint.com/java/

8.Oracle JDBC Documentation https://docs.oracle.com/javase/tutorial/jdbc/

THANK YOU